

Security Protocols and Their Analysis





SECURITY PROTOCOLS

A **security protocol** is a set of rules, followed by the communication parties during message exchange.

Its purpose is to achieve security goals such as:

- Authentication
- Key establishment
- Confidentiality
- Integrity

Example: the Diffie-Hellman protocol establishes a shared secret key

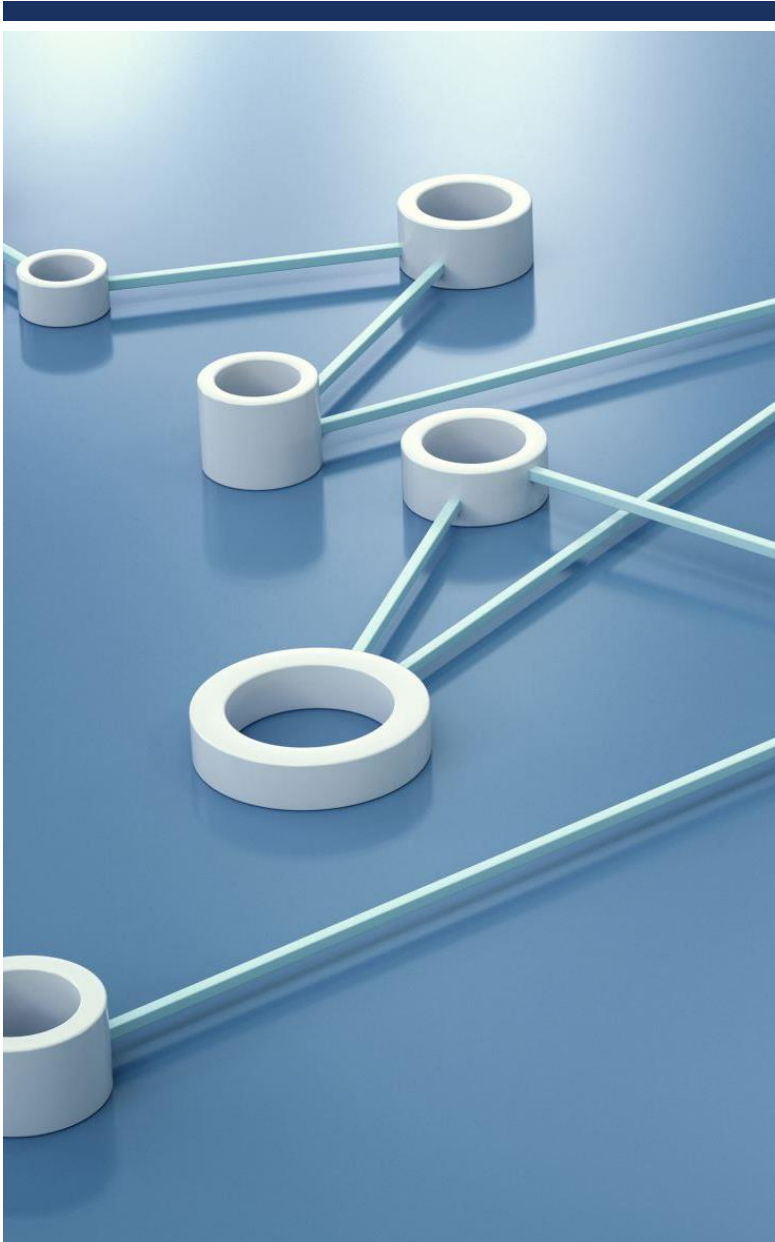
EXAMPLE: NEEDHAM-SCHROEDER PROTOCOL

The goal of the protocol is to establish **mutual authentication** between two parties A and B in the presence of adversary, who can

- Intercept messages
- Delay messages
- Read and copy messages
- Generate messages

But who does not know

- secret keys of parties, which they share with the authentication server S
- A and B obtain a secret shared key through authentication server S
- The protocol uses shared keys encryption/decryption



NEEDHAM-SCHROEDER PROTOCOL

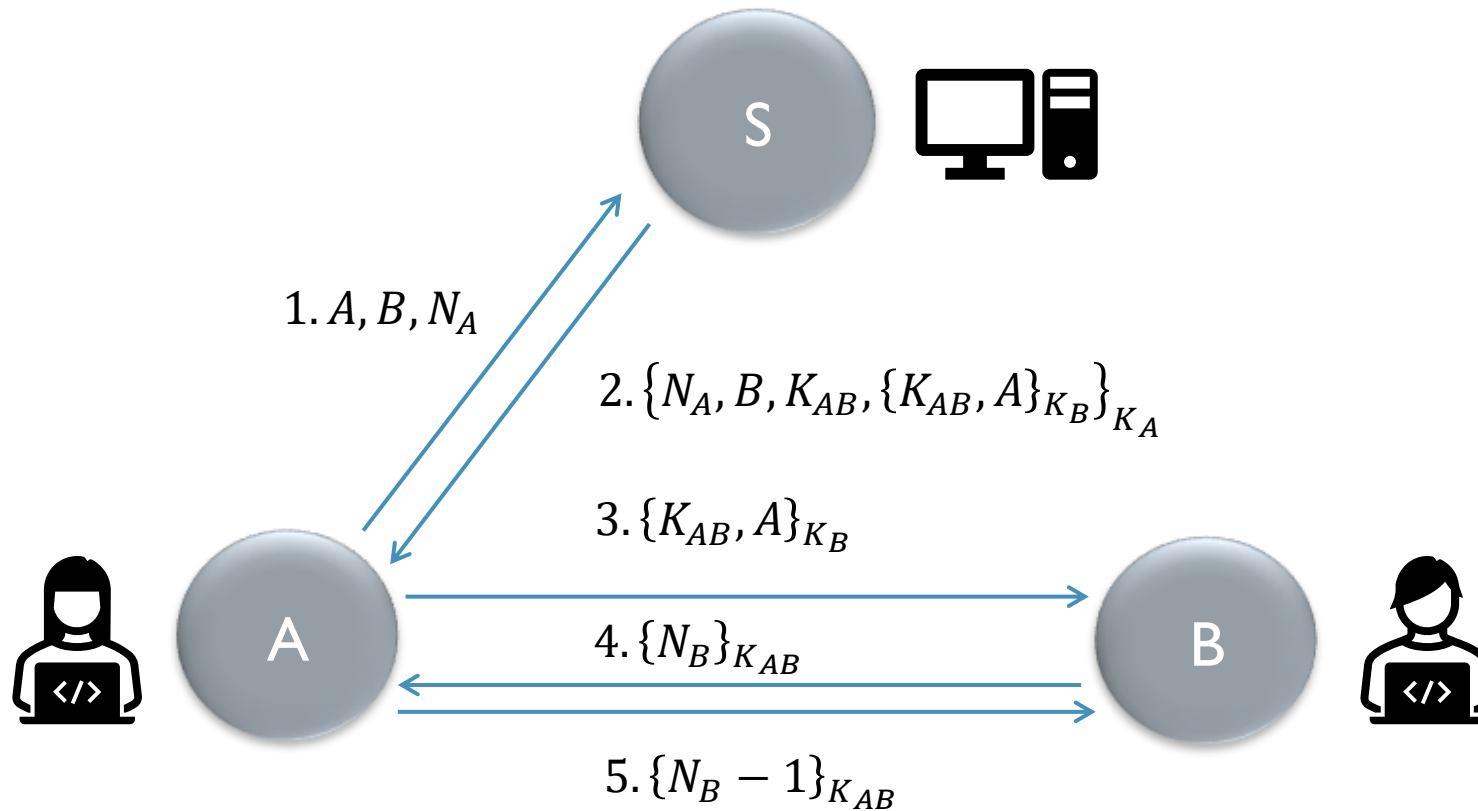


FIG: NEEDHAM-SCHROEDER PROTOCOL WITH SHARED KEYS

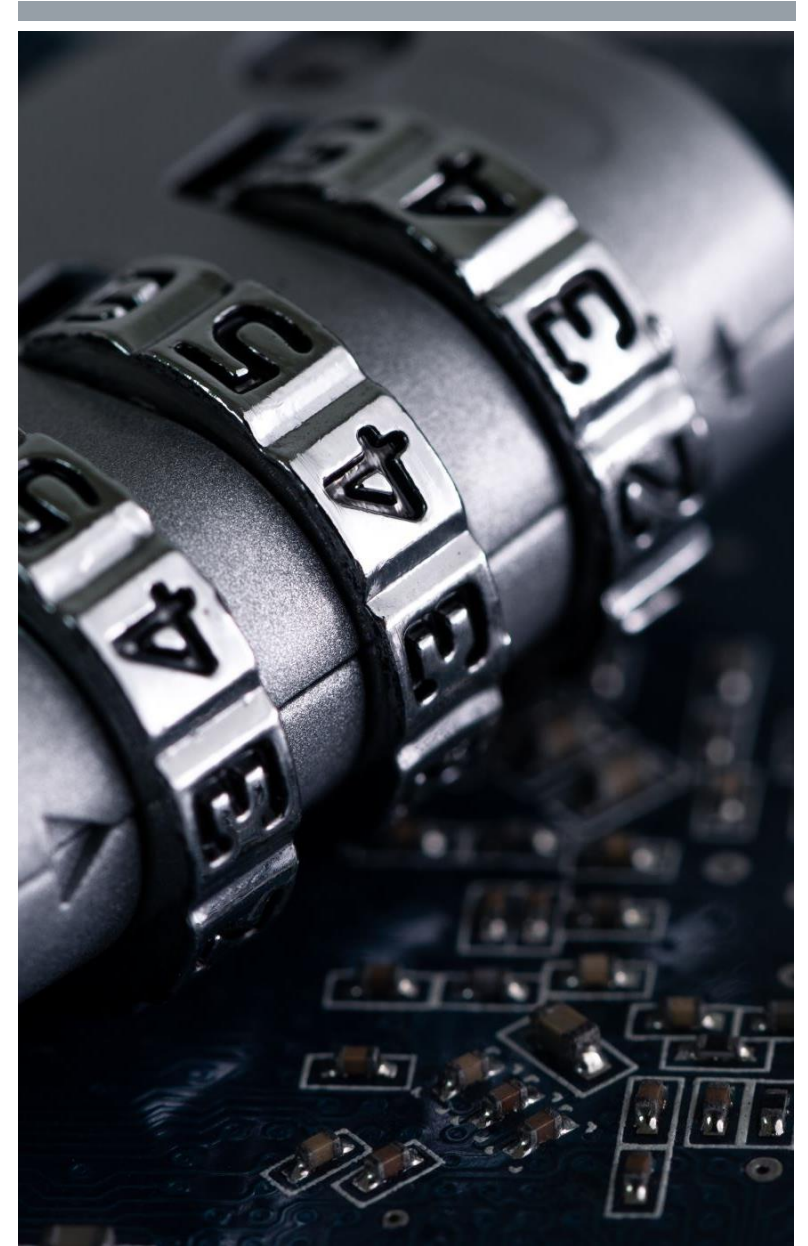
NEEDHAM-SCHROEDER PROTOCOL

- Message 1 $A \mapsto S: A, B, N_A$
 - Message 2 $S \mapsto A: \{N_A, B, K_{AB}, \{K_{AB}, A\}_{K_B}\}_{K_A}$
 - Message 3 $A \mapsto B: \{K_{AB}, A\}_{K_B}$
 - Message 4 $B \mapsto A: \{N_B\}_{K_{AB}}$
 - Message 5 $A \mapsto B: \{N_B - 1\}_{K_{AB}}$
-
- Here K_A and K_B are **keys** of A and B **shared with** S, resp.
 - N_A and N_B are **nonces**, introduced by A and B, resp.
 - K_{AB} is a **secret session key** for A and B provided by S

HOW IT WORKS

- A contacts the authentication server S, sending identities A and B and *nonce* N_A
- S responds with a message encrypted with the key of A
The message contains **session key** K_{AB} (to be used by A and B) and **certificate** encrypted with B's key conveying the session key and A's identity
- A sends the certificate to B
- B decrypts the certificates and sends his own nonce encrypted by the session key to A (*nonce handshake*)
- A decrypts the last message and sends *modified nonce* back to B

By the end of the message exchange both A and B share the secret key and both are assured in the presence of each other



CORRECTNESS OF PROTOCOLS

- Are they correct at all?
- A protocol may look correct, but still contain subtle flaws
- We have used semi-formal arguments, like

If a message is encrypted with the public key of Alice, then only a participant who knows private key of Alice (presumably Alice herself only) can decrypt it.

- Typically, we have considered possible attacks and argued using the reasoning as above, that attacks are impossible (under some reasonable assumptions).
- Is that enough? Are we sure that we have considered all possible situations of use?
- Formal analysis helps us check all possible behaviours more systematically



HOW TO ANALYSE A SECURITY PROTOCOL



What is each message carrying? (e.g. key, nonce, identity, encrypted data)



What does the receiver learn? (can they verify sender or just read data?)



What is the goal? (authentication, key exchange, confidentiality)



Can an attacker interfere? (intercept, replay, modify, impersonate)



Does encryption alone guarantee security? (check identity, freshness, session binding)

Key idea: A protocol may look secure step-by-step but fail when an attacker interacts with it.

TYPICAL ATTACKS TO LOOK FOR



Man-in-the-Middle (MITM):
attacker relays messages
between parties



Replay attack: reuse of old
messages (no freshness)



Impersonation: attacker
pretends to be another
party



Parallel session / reflection:
reuse messages across
sessions



Message substitution:
attacker modifies messages

Warning: Encryption alone is not enough — ensure identity, freshness, and correct session binding.

CORRECTNESS OF PROTOCOLS. II



Security protocols are designed to succeed even in the presence of malicious agents, often called *intruder (adversary)*



Intruder may have complete or partial control over the communication network and may have different computational capabilities



The correctness of the protocols depends on the assumptions on capabilities of possible intruder



Assumptions are often left implicit



Typically, in security we have to deal with numerous non-trivial assumptions

THE POWER OF FORMAL METHODS

What should we do about establishing correctness of security protocols?

Apply formal methods!

- Make **explicit** all the assumptions involved in a protocol
- Make a formal model of the protocol (and its execution)
- Apply formal reasoning, which would establish the correctness of the protocol

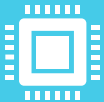
Two important aspects:

- The correctness is established only for a particular formal model of the protocol
- and under explicit assumptions (about capabilities of participants, etc)

LOGICAL REPRESENTATION



Formal aspects of reasoning is an important part of logic



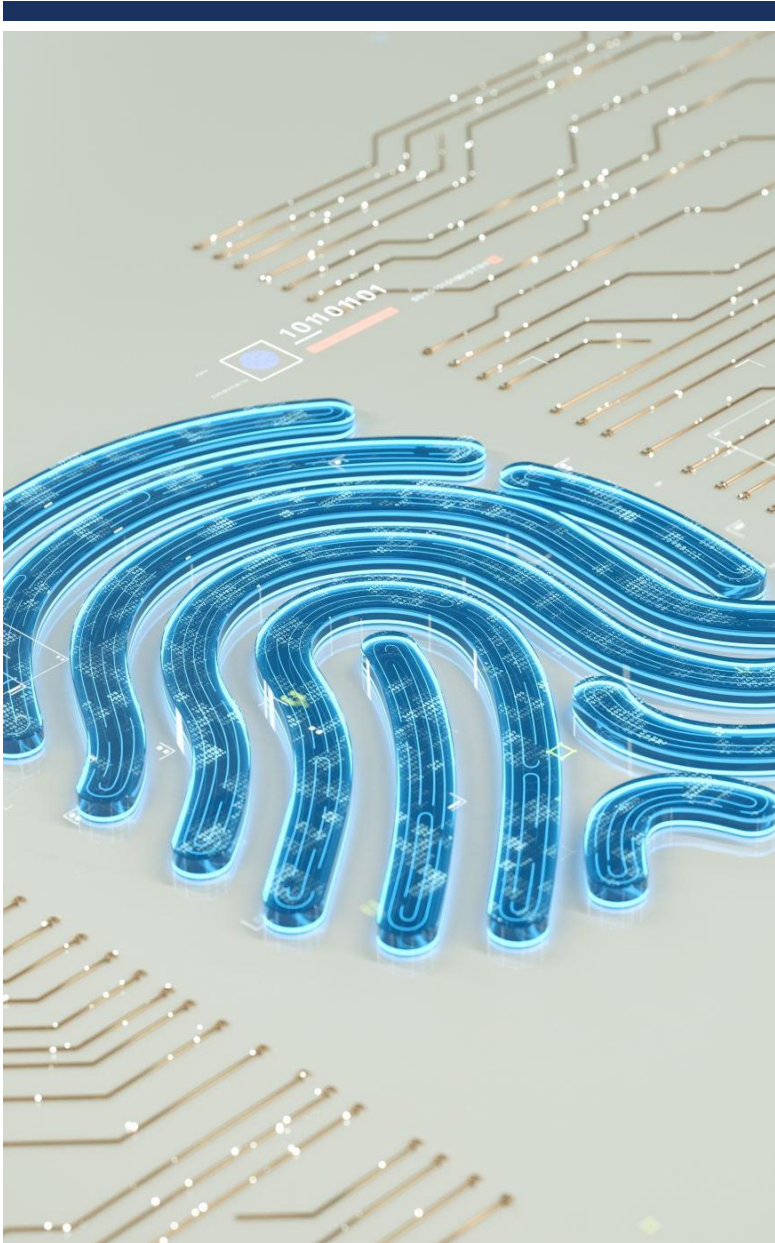
Logical representation and analysis of the security protocols is a particular successful approach for the protocol's verification



Non-classical modal epistemic logics dealing with such notions as “*belief*” and “*knowledge*”, are more suitable here than classical logics dealing primarily with “*truth*”

AUTOMATED VERIFICATION / ANALYSIS

- It is not easy and is error-prone to do formal analysis manually
- Development of methods for automated or semi-automated (interactive) validation and verification is important area, especially in the context of security protocols



DIFFERENT DIRECTIONS



Model checking (*try all possible executions*)

- specific (NRL Protocol Analyser, etc)
- general purpose tools (SMV, SPIN, etc)
- general purpose tools combined with specific translators (Casper, etc)
- Unbounded model checking for crypto protocols (ProVerif, etc)

Theorem proving (*prove no attack exists mathematically*)

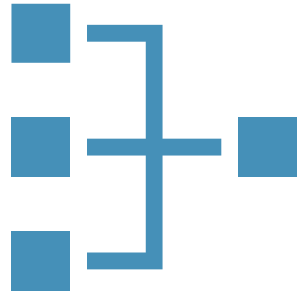
- Automated (TAPS, etc)
- Interactive (Isabell, PVS, etc)

Combinations of above techniques:

- Athena, etc

Others: decision procedures for specific theories, infinite state model checking, etc

GENERAL QUESTIONS



How to represent a protocol
(system) to be analysed?



How to express properties to be
verified?

MODEL CHECKING

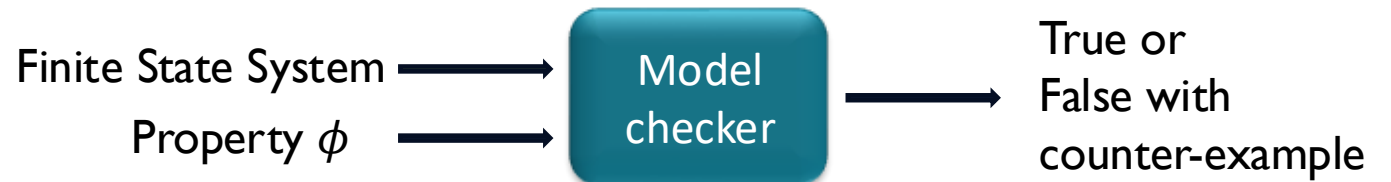
A protocol (system executing a protocol) is represented as a transition system $\mathbf{M}$ with **finitely** many states

A property to be analysed is expressed by a formula of a logic (temporal, modal, etc) f

Then verification amounts to checking whether the formula f is true in $\mathbf{M}$

Model checking is done via efficient state exploration techniques

MODEL CHECKING



Nice properties

- Fully automated procedures
- Very efficient state exploration

but

- Finite state abstraction is not always adequate, especially for protocols with unbounded number of participants or unbounded number of rounds.

ATTACK ON NEEDHAM- SCHROEDER PROTOCOL

A particular success of model checking methods in security protocol verification was discovery of a **flaw in NS protocol** based on public key cryptography (**Gavin Lowe**, 1995-1996)

Original protocol

Message 1: $A \rightarrow B: A, B, \{A, N_A\}_{PK(B)}$

Message 2: $B \rightarrow A: B, A, \{N_A, N_B\}_{PK(A)}$

Message 3: $A \rightarrow B: A, B, \{N_B\}_{PK(B)}$

ATTACK ON NEEDHAM-SCHROEDER PROTOCOL

Original protocol

Message 1: $A \rightarrow B: A, B, \{A, N_A\}_{PK(B)}$
Message 2: $B \rightarrow A: B, A, \{N_A, N_B\}_{PK(A)}$
Message 3: $A \rightarrow B: A, B, \{N_B\}_{PK(B)}$

Attack

Message 1a: $A \rightarrow I: A, I, \{A, N_A\}_{PK(I)}$
Message 1b: $I_A \rightarrow B: A, B, \{A, N_A\}_{PK(B)}$
Message 2b: $B \rightarrow I_A: B, A, \{N_A, N_B\}_{PK(A)}$
Message 2a: $I \rightarrow A: I, A, \{N_A, N_B\}_{PK(A)}$
Message 3a: $A \rightarrow I: A, I, \{N_B\}_{PK(I)}$
Message 3b: $I_A \rightarrow B: A, B, \{N_B\}_{PK(B)}$

Corrupt participant I impersonates A

THEOREM PROVING

- A protocol (a system) to be verified is described by a formula **Fs** of a logic (classical first-order, higher-order, modal, temporal, etc)
- A property to be verified is expressed by a formula **P** of the same logic
- Then to establish the required property it is enough to prove the theorem **Fs** $\rightarrow$ **P**



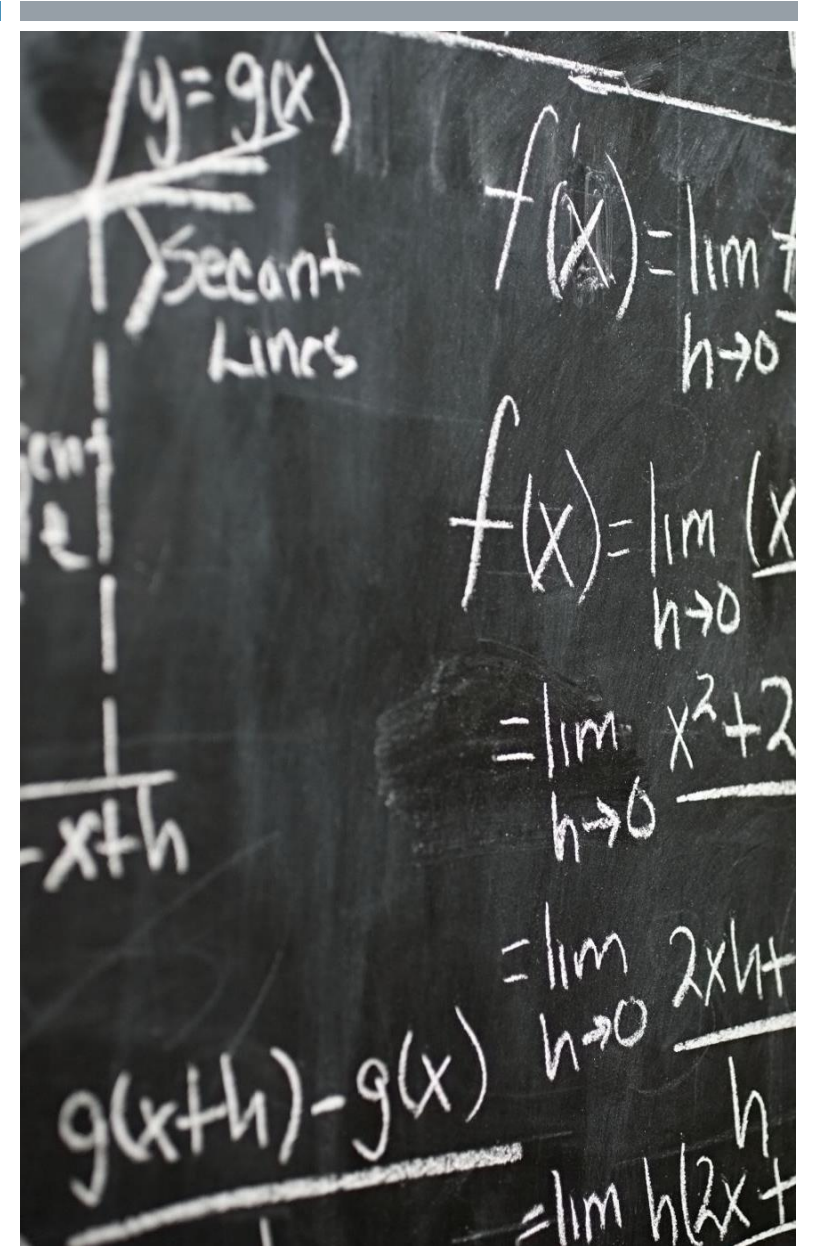
THEOREM PROVING

■ Potential benefits:

- the systems with *unbounded* (infinite) number states can be analysed
- E.g., “ $\forall x (S(x) \ \& \ B(x,m) \rightarrow R(x))$ ” may encode “any participant in a state S changes its state to R after sending message m”

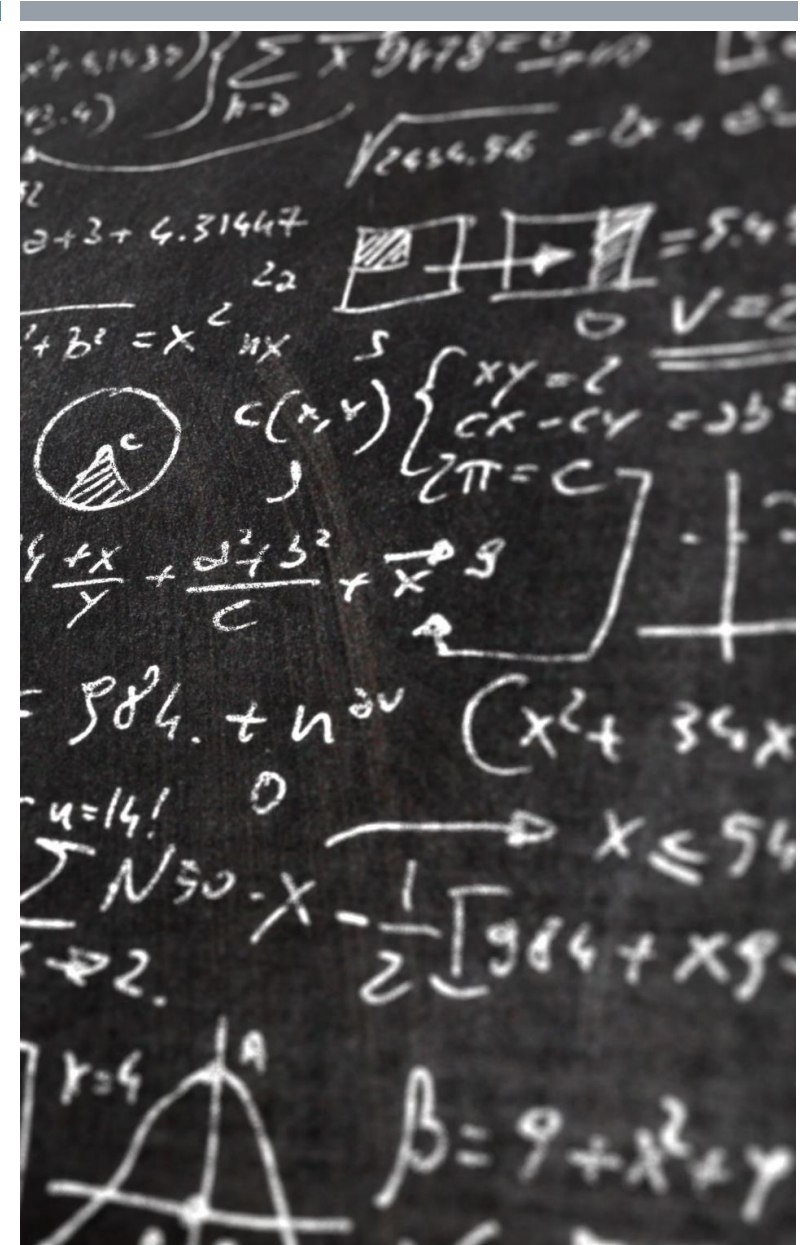
■ But:

- The problems here are, in general, *undecidable*
- Procedures are *incomplete* and of high complexity



THEOREM PROVING

- What to do?
- Apply automated procedures for fragments of first-order and higher-order logic
 - E. Cohen, TAPS system, Microsoft Research
- Use interactive theorem proving
 - L. Paulson, Cambridge: using Isabell, higher-order inductive theorem prover for the verification of security protocols
 - J. Bryans, S. Schenider, using interactive theorem prover PVS



SPECIALIZED APPROACHES

Bruno Blanchet, INRIA: approach based on ideas from Logic Programming

(ProVerif, available online at <https://bblanche.gitlabpages.inria.fr/proverif/>)

- A protocol is presented as a set of Horn clauses (like a program in Prolog), defining capabilities of all participants
- Verification then amounts to checking whether a security breaching goal can be derived from the set of clauses
- If the system detects the goal is unreachable, then *the protocol is correct*
- Standard operational semantics of Prolog is not very useful here due to undesirable looping
- Novel operational semantics (search strategy) is defined
- Recent versions use pi-calculus as a language for front-end

PROVERIF SYSTEM

Denning - Sacco key
distribution protocol

Message 1: $A \rightarrow B: \{\{k\}_{sk_A}\}_{pk_B}$

Message 2: $B \rightarrow A: \{s\}_k$

Explanation:

A generates session key k

A signs k with A's secret key

A encrypts the signed key with B's public key

B replies using k

Its representation in ProVerif system (old syntax)

Computation abilities of the attacker

pencrypt: $\text{attacker}(m) \wedge \text{attacker}(pk) \rightarrow \text{attacker}(\text{pencrypt}(m, pk))$

pk: $\text{attacker}(sk) \rightarrow \text{attacker}(pk(sk))$

pdecrypt: $\text{attacker}(\text{pencrypt}(m, pk(sk))) \wedge \text{attacker}(sk) \rightarrow \text{attacker}(m)$

sign: $\text{attacker}(m) \wedge \text{attacker}(sk) \rightarrow \text{attacker}(\text{sign}(m, sk))$

getmess: $\text{attacker}(\text{sign}(m, sk)) \rightarrow \text{attacker}(m)$

checksign: removed since implied by getmess

sencrypt: $\text{attacker}(m) \wedge \text{attacker}(k) \rightarrow \text{attacker}(\text{sencrypt}(m, k))$

sdecrypt: $\text{attacker}(\text{sencrypt}(m, k)) \wedge \text{attacker}(k) \rightarrow \text{attacker}(m)$

Initial knowledge of the attacker

$\text{attacker}(pk(sk_A)) \quad \text{attacker}(pk(sk_B)) \quad \text{attacker}(a[])$

Protocol

First message: $\text{attacker}(pk(r)) \rightarrow \text{attacker}(\text{pencrypt}(\text{sign}(k, sk_A[]), pk(r)))$

Second message: $\text{attacker}(\text{pencrypt}(\text{sign}(k, sk_A[]), pk(sk_B[]))) \rightarrow \text{attacker}(\text{sencrypt}(s[], k))$

KEY TAKE-AWAYS

- Protocols may look correct step by step yet still fail in the presence of an attacker.
- Encryption alone is not enough: protocols also need authentication, freshness, and correct session binding.
- Typical flaws include replay, impersonation, and man-in-the-middle attacks.
- Formal methods help us analyse protocols systematically, using model checking, theorem proving, and tools such as ProVerif.

A protocol is secure only if it remains secure even when an attacker can observe and interfere with the communication.

This is why protocol analysis matters.